

Alberta Buck – Externally-Driven Routing Simulation

Perry Kundert

2026-05-16

1 Why an Externally-Driven Simulation

Every BUCK simulation so far has lived **inside** a Forge test: the scenario, the actors, the clock, and the assertions are all Solidity, advanced by `vm.warp`. That was the right tool for validating the controller and the direct-mint mechanics in isolation. It is the wrong tool for the question now in front of us:

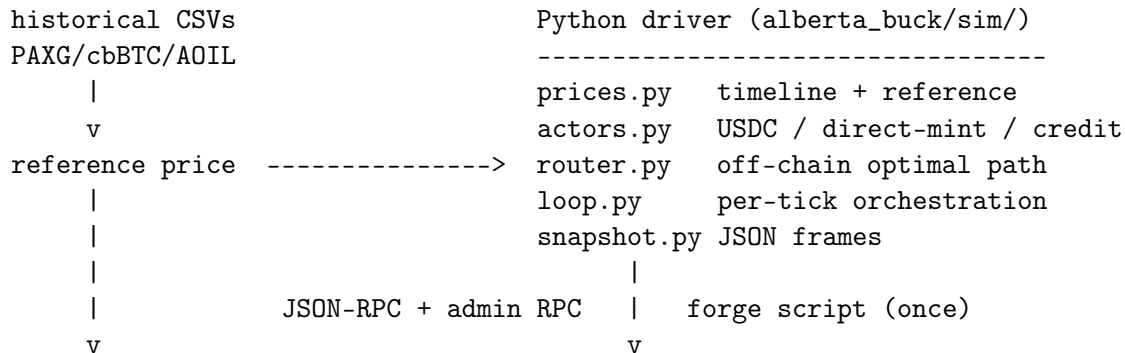
Does BUCK hold its peg over a multi-year historical price record, when a population of profit-seeking arbitrageurs routes freely across both the TOKEN/USDC pools and BUCK’s own pools using a real Uniswap router?

That question needs three things a Forge test cannot give:

1. **A real, multi-year exogenous price record** – actual PAXG, cbBTC and AOIL history fed in from CSV, not a synthetic drift.
2. **A real Uniswap execution layer** – the Universal Router, so the BUCK pools compete for routed flow on the same terms as the TOKEN/USDC pools, rather than a fixture’s hand-rolled swap.
3. **An external control loop** – a process that iterates the timeline, lets hundreds of actors make independent decisions per tick, and records the system’s internal state, without the gas and memory ceilings of a single in-process test transaction.

So the architecture inverts. Anvil becomes a **hosted EVM**, not a test runner. A Python program is the simulation: it owns the timeline, the actors, the price oracle, the pathfinding, and the plots. Solidity’s only remaining job is to be the BUCK system and the AMM, faithfully.

2 The Shape of the System



```

+-----+
|               anvil (hosted EVM)               |
|                                                 |
| IdentityRegistry  Buck  BuckCredit  BuckKControllerDirect |
| BuckBasket (+Receipt)                               |
| UniversalRouter (V4-era)  Permit2  WETH9           |
| UniswapV3Factory                               |
|   PAXG/USDC  cbBTC/USDC  AOIL/USDC   BUCK/USDC      |
|   PAXG/BUCK  cbBTC/BUCK  AOIL/BUCK                |
+-----+
|
| v
test/vectors/routing-sim.json --> test_routing_sim_plot.py --> images/routing-sim.png

```

Seven V3 pools. Four are the "truth" pools (TOKEN/USDC x3, BUCK/USDC). Three are the BUCK basket pools (TOKEN/BUCK x3) that the direct-mint path seeds. Every pool is Uniswap V3 because `BuckBasket` is V3-hardcoded; mixing AMM versions would strand the BUCK pools off the router.

3 The Routing Decision

The user added Uniswap's Universal Router. Two facts about it shape everything downstream.

3.1 Universal Router executes; it does not discover

`UniversalRouter.execute` is a loop over caller-supplied command bytes; `Dispatcher.dispatch` is a hardcoded `if/else` ladder that decodes each command's pre-encoded path and amounts. There is no pool enumeration, no quoting, no optimization on chain. **The caller must already know the route.**

This is not a deficiency – it is how every production Uniswap integration works. Route *discovery* is the job of the off-chain Smart Order Router; the on-chain router only *executes* the chosen path. Our topology is tiny and fixed (7 pools, ≤ 3 hops), so the discovery half is ~40 lines of Python that exhaustively quotes every candidate path and picks the best net-of-fee output. That Python **is** the routing algorithm, in the same sense that Uniswap's SOR is: it just doesn't need the SOR's generality for a 7-pool graph.

So the routing layer is two parts, not one:

- `alberta_buck/sim/router.py` – **discovery**. Enumerate paths, quote each (via an exact on-chain `eth_call` quoter), choose the best, encode the Universal Router command + path bytes.
- `UniversalRouter` on anvil – **execution**. Runs the encoded commands, custodies intermediate hops, settles against the pools.

The BUCK pools "are used by the router alongside the TOKEN/USDC pools" **endogenously**: once direct-mint arbers have seeded TOKEN/BUCK liquidity, `router.py` will select a BUCK-routed path whenever it yields more output. The route-usage histogram (see Plots) is the evidence that this is actually happening, rather than an assumption.

3.2 Universal Router is the V4-era router

The `lib/universal-router` sources are `pragma ^0.8.24` (pinned `solc 0.8.26, via_ir = true`). `UniversalRouter` unconditionally inherits `V4SwapRouter` -> `@uniswap/v4-periphery/V4Router`, which drags the entire Uniswap V4 core + periphery tree (~141 `.sol` files) into the build, plus `Permit2` (`pragma 0.8.17`). This is a real cost and the single biggest practicality risk in the whole plan – see Practicality.

We accept it because the user explicitly chose the Universal Router and the binding-makes-pools-router-operable model. But the design carries a documented fallback: a thin repo-authored `SimSwapper` + `SimQuoter`, extracted near-verbatim from the already-proven `test/fixtures/UniswapV3Fixture.sol` V3 swap callback. The off-chain `router.py` is identical in both worlds; only the execution contract differs. If the V4 compile tree proves intractable under this repo’s multi-solc Foundry setup, we swap the executor and lose nothing in the economic model.

4 The Permit2 / BUCK-approve Collision

Universal Router’s default input path pulls the user’s token through Permit2. Permit2 has exactly two on-chain touchpoints with any ERC20:

1. a one-time max ERC20 allowance from the holder to the Permit2 singleton (`token.approve(PERMIT2, type(uint256).max)`); Permit2 then meters per-spender, time-bounded sub-allowances internally via EIP-712 signatures;
2. `token.transferFrom(owner, recipient, amount)` invoked by Permit2 (`msg.sender = PERMIT2`) using the allowance from (1).

Permit2 never custodies the token – `transferFrom` moves it owner -> recipient atomically. So Permit2 requires **nothing of the pool**; it only needs (1) and (2) to behave like standard ERC20.

4.1 Where BUCK breaks each touchpoint

- **Touchpoint (1) is structurally forbidden.** `Buck.approve(address,uint256)` unconditionally `revert=s` (`=Buck.sol:219`); the only writer of `_allowances` is the 4-arg identity-bound `approve` (`Buck.sol:236`). There is no standard path to set `_allowances[actor][PERMIT2]`. This is the single fundamental incompatibility – everything else is satisfiable.
- **Touchpoint (2) already works – with one subtlety.** When Permit2 calls `BUCK.transferFrom(actor, pool, amt)`: `_spendAllowance(actor, PERMIT2, amt)` (`Buck.sol:757`) checks `_allowances[actor][PERMIT2]` a `type(uint256).max` allowance is **not decremented** (`Buck.sol:759`) – exactly Permit2’s expectation (the underlying ERC20 allowance to Permit2 is "infinite and untouched" while Permit2 meters sub-allowances). `_identityCheckedTransfer(actor, pool)` then needs `isVerified(actor)`, `isVerified(pool)`, and – since `_receiptFragments[actor][pool]` is zero – one side `isPublicIdentity` (`Buck.sol:599`). A public-bound pool satisfies this.

4.2 What each party must satisfy

- **The account (actor):** `isVerified`; an ERC20 allowance `_allowances[actor][PERMIT2] = max`; and an off-chain Permit2 EIP-712 `PermitSingle` signature granting the spender (Universal Router) a sub-allowance (pure Permit2-internal accounting – no BUCK/identity interaction, signed with the actor’s key).

- **The pool:** nothing is required by **Permit2** (Permit2 never touches it). BUCK's own `transferFrom` gate needs it `isVerified + isPublicIdentity` – the assumed public binding covers it. Its Carrying flag governs demurrage accounting on received BUCK; not a Permit2 concern.
- **Permit2 itself:** never holds BUCK and is never `from/=to`, so its Carrying flag is immaterial. It needs `isVerified` **only** on the heavyweight no-change route below (the 4-arg identity approve checks `isVerified(spender)`).

4.3 Two routes to BUCK-through-Permit2

- **Zero-change (the v1 design's fallback):** never use Permit2 for BUCK. The actor `transfer=s` BUCK into the Universal Router and calls `=execute` with `payerIsUser=false / CONTRACT_BALANCE`; the router pays the pool from its own balance via solmate `safeTransfer`. Permit2 untouched; `payerIsUser=true` stays dead for BUCK. A trailing `SWEEP / PAY_PORTION` command returns the output to the actor. USDC-only routes are unaffected (USDC is ungated); the driver standardises on pre-fund for uniformity.

```
payOrPermit2Transfer(token, payer, recipient, amount):
  if payer == address(this): pay() -> ERC20.safeTransfer (NO Permit2)
  else:                      PERMIT2.transferFrom      (BUCK: dead)
```

- **Minimal contract change (recommended for genuine compatibility):** a governance-set infrastructure allowlist that lets the **plain approve** succeed for whitelisted spenders only:

```
mapping(address => bool) public approvableInfra;          // governance-set
function approve(address spender, uint256 amount) external returns (bool) {
  require(approvableInfra[spender], "BUCK: use identity-bound approve");
  _approve(msg.sender, spender, amount);                  // standard ERC20 path
  return true;
}
```

Governance whitelists the Permit2 singleton (and optionally the Universal Router). Ordinary counterparties still **must** use the identity-bound approve – the privacy/receipt model is unchanged for person-to-person payments. Only sanctioned, audited routing infrastructure gets the standard ERC20 path. `transferFrom` is unaffected: it already does the right thing (identity-checked actor -> pool, public pool satisfies it, max allowance non-decrementing per Permit2's expectation). ~6 lines + a governance setter makes BUCK a first-class Permit2 (hence Universal Router, hence any standard router) citizen without weakening the counterparty-privacy invariant. With this route Permit2 need **not** be identity-bound at all (the allowance is set via the plain path, so `isVerified(PERMIT2)` is never checked), and the full `payerIsUser=true` UR command set becomes available for BUCK – removing the pre-fund choreography entirely.

4.4 Caveat: the receipt-fragment relaxation

Under the allowlist route the actor -> pool BUCK transfer is gated purely by the public-pool rule; no per-(actor,pool) receipt fragment exists, because the approval was actor -> Permit2, not actor -> pool. Permit2-routed BUCK movements therefore bypass the per-counterparty receipt-fragment trail that direct identity-bound transfers leave. For routing infrastructure against public-bound pools that is the intended trade-off – a deliberate relaxation, not an oversight.

5 Identity Binding

BUCK transfer requires `isVerified(from)` AND `isVerified(to)`, plus at least one side `isPublicIdentity` (absent a per-pair receipt). For any route that touches BUCK, every address BUCK rests in or moves through must be `bindContract`-bound Public+Carrying:

- **The Universal Router contract itself** – it is pre-funded with BUCK and pays pools from its own balance, and custodies intermediate BUCK between multi-hop V3 legs. Mandatory.
- **Every BUCK-containing V3 pool** – the 3 TOKEN/BUCK pools and the BUCK/USDC pool. `BuckBasket` does NOT self-bind the pools it creates; the deploy step binds each one explicitly.
- **Every actor that holds or sends BUCK** – direct-mint and credit-mint arbers. Pure-USDC arbers never touch BUCK and need no binding.
- `BuckBasket` – per the canonical wiring.
- **Permit2 binding depends on the route** (see the Permit2 section): under the zero-change pre-fund flow Permit2 is never touched and not bound; under the recommended `approvableInfra` allowlist change Permit2 still need not be bound (the allowance uses the plain `approve` path, so `isVerified(PERMIT2)` is never checked). Only the heavyweight no-change "4-arg approve to Permit2" variant would require binding Permit2 Public.

For the simulation we use the `anvil-admin` shortcut the test fixtures already sanction (`vm.etch` + `bindContract` in tests; the RPC analogues here):

- `anvil_setCode(addr, "0x00")` to satisfy `bindContract`'s `code.length > 0` for actor EOAs and helper contracts, then `IdentityRegistry.bindContract(addr, G1(), {R:G1(),C:G1()}), isPublic=true, isCarrying...)=`.
- The subtlety: **an account with code cannot be a normal tx sender**. Actor BUCK movements are therefore driven through bound contract proxies / `anvil_impersonateAccount`, decided concretely in the Phase 2 slice. USDC-only actors stay codeless EOAs and transact normally.
- BUCK allowances (only if a non-pre-fund path is ever needed) via `anvil_setStorageAt` at `keccak256(spender . keccak256(owner . p))` where `p` is read from `forge inspect Buck storageLayout` – never hardcoded.

We use the shortcut because this simulation's subject is price and controller dynamics, not the identity protocol. Faithful ZK registration would require issuer-signed credentials per actor; only `alice/=bob` vectors exist.

6 Deployment

`script/Deploy.s.sol` is the wrong template – it deploys the legacy USD-intermediated controller and never wires the basket. The canonical 12-step Direct-system wiring lives in `test/BuckBasket.t.sol::setUp()`.

A new `script/ScenarioDeploy.s.sol` transcribes that sequence and adds the Universal Router stack:

1. `IdentityRegistry(gov); gov.trustIssuer.`
2. `BuckCredit().`
3. `BuckKControllerDirect(8-arg).`
4. `Buck(buckCredit, kCtrl, identity, insurancePool).`
5. `identity.setBuck(buck).`
6. `UniswapV3Factory` (deployCode from out/ artifact).
7. `Permit2` (deployed even though BUCK never uses it; UR's constructor consumes `RouterParameters.permit2`).
8. `WETH9; UniversalRouter(RouterParameters{permit2, weth9, v2Factory=0, v3Factory, poolInitCodeHash, v4PoolManager..., ...})=`. V4 fields wired but no `V4_SWAP` command is ever issued.
9. `BuckBasket(buck, kCtrl, v3Factory, gov, feeTier, twapWindow, cardinality, maxDevBp, minSeedL).`
10. `buck.setBasket(basket) FROM insurancePool.`
11. `kCtrl.setBasket(basket).`
12. `identity.bindContract(basket, Public, Carrying); identity.bindContract(universalRouter, Public, Carrying).`
13. Per token: `basket.addBasketToken(token, dec, initPrice, weightBp, feeTier)` then `identity.bindContract(returnedPool, Public, Carrying).`
14. Create the 4 TOKEN/USDC + BUCK/USDC V3 pools, seed initial liquidity, bind the BUCK/USDC pool.

Scenario parameters (token list, initial prices, fee tier, TWAP window, cardinality, equal 1/3 weights) are passed to the script via a JSON config it `vm.readFile=s`. The driver invokes the script once: `=forge script ScenarioDeploy --broadcast --rpc-url <anvil>`, then parses `broadcast/.../run-latest.json` into a typed `Deployment` dataclass. Actor `identity/credit` setup (dynamic, numerous) is the driver's job afterward, not the script's.

7 The External Driver

A new Python package `alberta_buck/sim/` (single language end-to-end – `web3.py` and `matplotlib` are already project deps; the plot pipeline is already `pytest/matplotlib`):

- `anvil.py` – process lifecycle + admin RPC: `set_code`, `set_storage_at`, `set_next_block_timestamp`, `mine`, `impersonate`. The time-control layer that replaces `vm.warp`.
- `deploy.py` – runs `ScenarioDeploy` once, parses addresses.
- `prices.py` – CSV loader; per-token multi-year series aligned to a common tick grid; `reference_price(token, t)`.

- `router.py` – off-chain path discovery: enumerate the fixed graph, quote each hop via an exact `eth_call` quoter, return and encode the best Universal Router command sequence; route-usage counter.
- `actors.py` – `UsdcArber`, `DirectMintArber`, `CreditMintArber`; Gaussian arrival + threshold-arb policy ported from `SimulationFixture.sol` semantics; the `AdoptionSchedule`.
- `snapshot.py` – per-tick state reader + JSON writer in the established `test/vectors/*.json` schema.
- `loop.py` – the timeline iterator and per-tick orchestration.
- `__main__.py` – CLI: `python -m alberta_buck.sim --config ....`

7.1 Per-tick sequence (order is load-bearing)

1. `anvil_setNextBlockTimestamp(t)`; `evm_mine` # advance the clock
2. touch all 7 pools (write a TWAP observation at `t`) # keep `consult()` sane
3. `prices.reference_price(token, t)` for each token # update truth
4. actor decision pass:
 - each active actor: compare pool implied price (router quote)
 - vs reference; size a trade that approaches -- not overshoots --
 - equilibrium; choose mechanism per `AdoptionSchedule`
5. execute:
 - `UsdcArber` -> `UniversalRouter (USDC<->TOKEN, pre-fund flow)`
 - `DirectMintArber` -> `BuckBasket.depositToken` -> seed `TOKEN/BUCK`,
 - then route the spread via UR through `BUCK` pools
 - `CreditMintArber` -> `BuckCredit createCredit/activate` -> `Buck.mint`
 - > arb with `BUCK` via UR
6. `kCtrl.compute()` # keep PID warm
7. actor lifecycle + adoption-fraction roll-forward
8. `snapshot.capture(t)` # append a frame

8 Reference-Price Arbitrage Model

For each `TOKEN` per tick: `edge = (reference - poolImplied) / reference`. `poolImplied` is the best multi-hop quote from `router.py` (so `BUCK` pools enter the comparison once seeded). If `edge > +threshold` the token is cheap on chain -> buy `TOKEN`, pushing pool price up toward reference. If `edge < -threshold` -> sell. Trade size is the input that would move spot to $\sim$ reference (solved against the quoter), scaled by an aggressiveness factor < 1 so equilibrium is approached rather than bang-banged – the Alice threshold policy from `SimulationFixture.sol` (entry $\sim 3\%$, exit $\sim 0.5\%$, fractional deployment), ported.

AOIL ("Alberta Oil") is synthetic: a plain mock ERC20 whose only "truth" is its CSV. `PAXG` and `cbBTC` are treated identically for the sim – the CSV is ground truth, the pools are arbitrated toward it. Equal basket weights (1/3 each).

8.1 Adoption schedule

A piecewise/logistic ramp parameterised in the scenario config maps timeline progress to the mechanism mix of **newly arriving** arbers:

- Early: ~100 % `UsdcArber` (TOKEN/USDC only; never touches BUCK; no binding needed).
- Mid: rising `DirectMintArber` – pledges TOKEN into `BuckBasket.depositToken`, gets a `BuckBasketReceipt`, the minted BUCK seeds the TOKEN/BUCK pool, profit shared with the Buck contract per the basket mechanics. These actors are identity-bound.
- Late: a slice of `CreditMintArber` – BUCK minted against insured `BuckCredit` assets, used to arb.

The user can tune the curve and re-run; the ramp is the independent variable in the headline experiment.

9 Snapshots and Plots

One JSON frame per tick, schema modelled on `equilibrium-scenario.json` (parallel per-field arrays). Captured per tick:

- Per pool (all 7): `spot`, `twap`, and the CSV reference for the underlying; pool liquidity (slot0 L) for depth.
- Controller: `buckK`, PID `pid_p/pid_i/pid_d`, `fundingFactor`, `basketValueInBuck`.
- BUCK aggregate: `totalSupply`, `jubilee`.
- Actors: active/retired counts per class; cumulative realized PnL per class.
- `route_usage`: histogram {canonical-path -> count} since last snapshot – the direct evidence BUCK pools entered routing.

A new `alberta_buck/test/test_routing_sim_plot.py` (reusing the exact `pytest.mark.skipif + images/` convention) renders a six-panel figure; one new `make plots` line:

1. **TOKEN/USDC vs reference** – do arbers hold the pools to truth.
2. **BUCK equilibrium** – BUCK/USDC `spot+twap` vs \$1.00 and `basketValueInBuck` (the headline success metric).
3. **Controller internals** – `buckK` + `fundingFactor`, symlog P/I/D sub-panel (reuse the existing symlog treatment).
4. **Supply & populations** – `totalSupply` + active counts per class, adoption ramp visible.
5. **Route usage** – stacked area over time (proof BUCK pools are routed alongside TOKEN/USDC).
6. **Per-class PnL** – validates the economic incentive story.

10 Practicality

10.1 Straightforward (reuse)

The Python skeleton, CSV loading, snapshot/JSON, and matplotlib are pure reuse of established repo patterns. The off-chain router is tiny and exact. `ScenarioDeploy.s.sol` is a transcription of an existing `setUp()`. The actor policy logic is a port of proven semantics.

10.2 The heavy lifting

- **Compiling the V4-era Universal Router in this repo.** ~141 V4 files + Permit2 (0.8.17) + UR (0.8.26, `via_ir`) added to a Foundry project already juggling solc 0.5.16/0.6.6/0.7.6/0.8.31 via compile-trigger shims. Remappings must be repointed (UR expects `@uniswap/v3-core` at `node_modules/`, this repo has `lib/v3-core`). This is the dominant risk; the `SimSwapper` fallback exists precisely for it.
- **The actor model end-to-end against live anvil** (vs in-process Foundry) – the bulk of the engineering.
- **The code-vs-sender identity tension** – a bound account has code and cannot be a normal tx origin; BUCK-moving actors route through bound proxies / impersonation.

10.3 Biggest risks (ranked)

1. **Universal Router V4 compile tree** – may force the `SimSwapper` fallback. Decided by a Phase-2 spike, before any economics depend on it.
2. **TWAP warmup over RPC across multi-year timelines** – every pool touched every tick for the whole record; cardinality sized to the tick spacing or observations wrap. Phase-0 asserts `consult()` is non-degenerate before any economics are trusted.
3. **CSV cadence vs block cadence vs TWAP window** – `tick_seconds` must satisfy `tick_seconds > twap_window / cardinality=`; the config couples them; the CSV is downsampled to that grid.
4. **Anvil state growth over tens of thousands of ticks** – incremental JSON flush, `--prune-history`, tick count tunable so a 6-month slice validates before a 5-year run.
5. **Identity friction** – bounded and mechanical given the shortcut.

10.4 Phased build order (smallest vertical slice first)

- **Phase 0** – skeleton + 1 TOKEN/USDC pool, no BUCK, no UR (direct pool swap). Proves time-control + swap + snapshot + plot loop. Gate: pool tracks CSV; `consult()` non-degenerate.
- **Phase 1** – 3 TOKEN/USDC pools + off-chain pathfinder + UR (the V4 compile spike happens here; fall back to `SimSwapper` if needed). Gate: all 3 pools track; router picks correct routes.
- **Phase 2** – full Direct system + identity wiring + the BUCK pre-fund / `payerIsUser=false` path; add `DirectMintArber`. Gate: a BUCK route executes; one direct-mint round-trip profits; `basketValueInBuck` sane.
- **Phase 3** – adoption schedule + `CreditMintArber` + controller in the loop, multi-hop through BUCK pools. Gate: BUCK holds ~\$1.00 over a 6-month CSV slice; route-usage shows BUCK pools entering as adoption rises.
- **Phase 4** – full multi-year run; finalize the six-panel figure; expose scenario params for tuning.

10.5 Verdict

Buildable. The economic model and the off-chain router are easy and mostly reuse. The one genuine gamble is the V4-era Universal Router compile; the design neutralises it with a drop-in fallback that changes nothing above the execution contract. The user's peg question gets a real, multi-year, router-mediated answer.

11 Out of Scope (v1)

- V4 swaps (UR can, we never issue `V4_SWAP`).
- Real ZK identity registration for actors (anvil-admin shortcut).
- Permit2 for BUCK via the **standard** path is gated on the optional **approvableInfra** allowlist change (see the Permit2 section). The v1 zero-change design routes BUCK by pre-funding the router instead; adopting the ~6-line allowlist promotes BUCK to a first-class Permit2/router citizen and is the recommended follow-on. USDC may use Permit2 freely either way.
- Basket rebalancing / fee-tier migration (inherited v2 items from the direct embodiment).
- On-chain route discovery (off-chain by design, as in production).